STACK

Complete GATE DA Study Notes

Theory + Diagrams + Algorithms + Python + Numericals

1 What is a Stack?

Definition

A **Stack** is a linear data structure in which insertion and deletion are performed only at one end, called the **TOP**.
Stack follows the principle:

$$\boxed{\text{LIFO} = \text{Last In, First Out}}$$

The element inserted last is removed first.

Simple Example

Suppose we push:

$$10,\ 20,\ 30,\ 40$$

The stack becomes:

$$\boxed{40}$$

$$\boxed{30}$$

$$\boxed{20}$$

$$\boxed{10}$$

The element 40 is at the top.
Therefore:

$$\boxed{\text{POP} \rightarrow 40}$$

2 Real-Life Examples of Stack

Real-Life Examples

- Stack of plates
- Stack of books
- Browser back button
- Undo operation
- Function call stack
- Expression evaluation
- Parentheses matching
- Depth First Search (DFS)

The last plate placed on a stack is the first plate removed.
Hence:

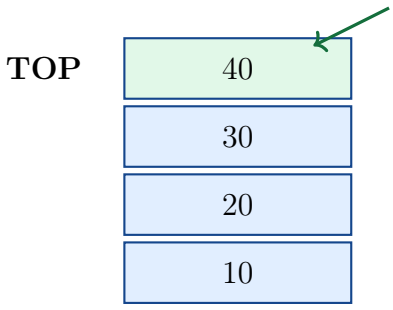
$$\boxed{\text{LIFO}}$$

3 Stack Diagram

Consider:

$$10,\ 20,\ 30,\ 40$$

where 40 is the top element.



Therefore:

$$\boxed{TOP = 40}$$

4 Basic Stack Operations

The main stack operations are:

1. PUSH
2. POP
3. PEEK / TOP

4. isEmpty
5. isFull

5 PUSH Operation

PUSH

PUSH inserts a new element at the top of the stack.

$$\boxed{\text{PUSH}(x)}$$

adds x to the TOP.

Suppose:

30
20
10

Push:

40

Result:

40
30
20
10

Therefore:

$$\boxed{TOP = 40}$$

6 PUSH Algorithm – Array Implementation

Assume:

$$TOP = -1$$

for an initially empty stack.
For a stack of capacity N:

PUSH Algorithm

1. Check whether the stack is full.
2. If full, report **Stack Overflow**.
3. Otherwise increment TOP.
4. Store the new element at $STACK[TOP]$.

Pseudocode:

```
PUSH(STACK, x)

if TOP == N-1
    Overflow
else
    TOP = TOP + 1
    STACK[TOP] = x
```

7 POP Operation

POP

POP removes the element from the top of the stack.

$$\boxed{\text{POP}()}$$

returns and removes the current top element.

Example:
Before POP:

40
30
20
10

After POP:

30
20
10

Removed element:

$$\boxed{40}$$

8 POP Algorithm

POP Algorithm

1. Check whether the stack is empty.
2. If empty, report **Stack Underflow**.
3. Otherwise read $STACK[TOP]$.
4. Decrement TOP.
5. Return the removed value.

Pseudocode:

```
POP(STACK)

if TOP == -1
    Underflow
else
    x = STACK[TOP]
    TOP = TOP - 1
    return x
```

9 PEEK / TOP Operation

PEEK returns the top element without removing it.

$$\boxed{\text{PEEK()}}$$

Example:

$$\begin{array}{c} 50 \\ 40 \\ 30 \\ 20 \end{array}$$

PEEK returns:

$$\boxed{50}$$

The stack remains unchanged.

Important Difference

$$\boxed{\text{PEEK} \rightarrow \text{read only}}$$

$$\boxed{\text{POP} \rightarrow \text{read} + \text{remove}}$$

10 Stack Overflow and Underflow

10.1 Stack Overflow

Overflow occurs when we try to PUSH into a full stack.

For capacity:

$$N = 5$$

if:

$$TOP = 4$$

then the stack is full.

Another PUSH causes:

$$\boxed{\text{STACK OVERFLOW}}$$

10.2 Stack Underflow

Underflow occurs when we try to POP from an empty stack.

For an empty array-based stack:

$$TOP = -1$$

Therefore:

$$\boxed{\text{POP from empty stack} = \text{UNDERFLOW}}$$

11 Stack States

For an array stack of capacity N:

$$\boxed{TOP = -1}$$

means:

$$\boxed{\text{Empty}}$$

and:

$$\boxed{TOP = N - 1}$$

means:

$$\boxed{\text{Full}}$$

Number of elements currently present:

$$\boxed{TOP + 1}$$

provided:

$$TOP \geq -1.$$

12 Important Stack Formula

For zero-based indexing:

$$\boxed{\text{Number of elements} = TOP + 1}$$

Therefore:
If:

$$TOP = 0$$

then:

$$1 \text{ element}$$

If:

$$TOP = 4$$

then:

$$5 \text{ elements}$$

If:

$$TOP = 9$$

then:

$$10 \text{ elements}$$

13 Stack Operation Tracing

Consider the operations:

$$PUSH(10)$$

$$PUSH(20)$$

$$PUSH(30)$$

$$POP()$$

$$PUSH(40)$$

$$POP()$$

Let's trace.

Step 1

$$PUSH(10)$$

Stack:

$$[10]$$

Step 2

$$PUSH(20)$$

Stack:

$$[10, 20]$$

TOP:

$$20$$

Step 3

$$PUSH(30)$$

Stack:

$$[10, 20, 30]$$

Step 4

$$POP()$$

Removed:

$$\boxed{30}$$

Stack:

$$[10, 20]$$

Step 5

$$PUSH(40)$$

Stack:

$$[10, 20, 40]$$

Step 6

$$POP()$$

Removed:

$$\boxed{40}$$

Final stack:

$$\boxed{[10, 20]}$$

14 Stack Using Python List

Python lists can implement stacks naturally.

```
stack = []

stack.append(10)
stack.append(20)
stack.append(30)

print(stack)

x = stack.pop()

print("Removed:", x)
print("Stack:", stack)
```

Output:

```
[10, 20, 30]
Removed: 30
Stack: [10, 20]
```

Therefore:

$$\boxed{\text{append}() = PUSH}$$

$$\boxed{\text{pop}() = POP}$$

15 Stack Class in Python

```python
class Stack:

    def __init__(self):
        self.items = []

    def push(self, x):
        self.items.append(x)

    def pop(self):
        if self.is_empty():
            return "Underflow"

        return self.items.pop()

    def peek(self):
        if self.is_empty():
            return "Stack is empty"

        return self.items[-1]

    def is_empty(self):
        return len(self.items) == 0

    def size(self):
        return len(self.items)


s = Stack()

s.push(10)
s.push(20)
s.push(30)

print(s.peek())
print(s.pop())
print(s.size())
```

16 Time Complexity of Stack Operations

Operation	Time	Extra Space
PUSH	$O(1)$	$O(1)$
POP	$O(1)$	$O(1)$
PEEK	$O(1)$	$O(1)$
isEmpty	$O(1)$	$O(1)$
isFull	$O(1)$	$O(1)$

GATE DA Important

For a standard stack implementation:

$$\boxed{PUSH = POP = PEEK = O(1)}$$

17 Stack Using Array

Consider:

$$N = 5$$

Initially:

$$TOP = -1$$

Array:

0	1	2	3	4
–	–	–	–	–

After:

$$PUSH(10)$$

$$TOP = 0$$

After:

$$PUSH(20)$$

$$TOP = 1$$

After:

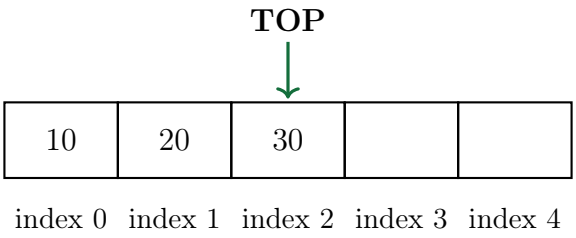
$$PUSH(30)$$

$$TOP = 2$$

Stack:

10	20	30	–	–
0	1	2	3	4

18 Array Stack Representation



Here:

$$TOP = 2$$

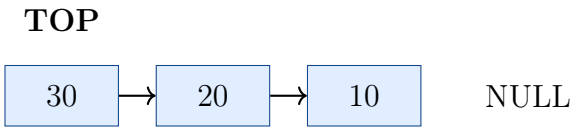
and number of elements:

$$TOP + 1 = 3.$$

19 Stack Using Linked List

A stack can also be implemented using a linked list.

The top of the stack is usually maintained at the head.



PUSH:

$$\boxed{O(1)}$$

POP:

$$\boxed{O(1)}$$

if TOP is maintained at the head.

20 Array Stack vs Linked-List Stack

Property	Array Stack	Linked-List Stack
Size	Usually fixed/dynamic array	Dynamic
PUSH	$O(1)$ amortized for dynamic array	$O(1)$
POP	$O(1)$	$O(1)$
Memory overhead	Low	Extra pointer required
Overflow	Capacity-dependent	Memory-dependent
Random access	Possible but not a stack operation	Possible by traversal only

21 Applications of Stack

Important Applications

1. Function calls
2. Recursion
3. Undo/Redo
4. Browser history
5. Parentheses matching
6. Infix to postfix conversion
7. Postfix evaluation
8. Prefix evaluation
9. Depth First Search
10. Backtracking

22 Function Call Stack

Consider:

```
def A():
    B()

def B():
    C()

def C():
    print("Hello")

A()
```

When A calls B:

A

B

When B calls C:

A

B

C

Since C finishes first:

$$C \to B \to A$$

This follows:

$$\boxed{\text{LIFO}}$$

23 Recursion Uses Stack

Consider factorial:

$$5! = 5 \times 4 \times 3 \times 2 \times 1$$

Recursive function:

```
def factorial(n):

    if n == 0:
        return 1

    return n * factorial(n-1)
```

Function calls:

$$factorial(5)$$

$$factorial(4)$$

$$factorial(3)$$

$$factorial(2)$$

$$factorial(1)$$

$$factorial(0)$$

Then return:

$$0 \to 1 \to 2 \to 3 \to 4 \to 5$$

The call stack follows LIFO.

24 Balanced Parentheses

A stack can check whether parentheses are balanced.

Example:

$$\boxed{(A + B) \times (C - D)}$$

is balanced.

But:

$$\boxed{(A + B \times (C - D)}$$

is not balanced.

Algorithm

1. Scan the expression from left to right.
2. If an opening bracket is found, PUSH it.
3. If a closing bracket is found:
   - If stack is empty, expression is invalid.
   - Otherwise POP and check whether the brackets match.
4. At the end, stack must be empty.

Supported brackets:

()

[]

{}

25 Balanced Parentheses – Python

```python
def is_balanced(expression):

    stack = []

    pairs = {
        ')': '(',
        ']': '[',
        '}': '{'
    }

    for ch in expression:

        if ch in "([{":
            stack.append(ch)

        elif ch in ")]}":

            if not stack:
                return False

            if stack.pop() != pairs[ch]:
                return False

    return len(stack) == 0

```

```
26
27 print(is_balanced("(A+B)"))
28 print(is_balanced("([A+B])"))
29 print(is_balanced("(A+B]"))
```

26 Expression Notations

There are three important notations:

1. Infix
2. Prefix
3. Postfix

Example:

$$A + B$$

Infix:

$$\boxed{A + B}$$

Prefix:

$$\boxed{+AB}$$

Postfix:

$$\boxed{AB+}$$

27 Postfix Expression Evaluation

Consider:

$$2\ 3\ +\ 4\ *$$

Algorithm:

1. Scan from left to right.
2. If operand, PUSH it.
3. If operator, POP required operands.
4. Perform operation.
5. PUSH result.

28 Postfix Numerical Example

Evaluate:

$$2\ 3\ +\ 4\ *$$

Step 1:

$$PUSH(2)$$

Stack:

$$[2]$$

Step 2:

$$PUSH(3)$$

Stack:

$$[2, 3]$$

Step 3:

$$+$$

POP:

$$3,\ 2$$

Calculate:

$$2 + 3 = 5$$

PUSH:

$$[5]$$

Step 4:

$$PUSH(4)$$

$$[5, 4]$$

Step 5:

$$*$$

$$5 \times 4 = 20$$

Therefore:

$$\boxed{20}$$

Important

For a binary operator:

$$op$$

first POP gives:

$$b$$

second POP gives:

$$a$$

Then calculate:

$$\boxed{a\ op\ b}$$

Do not reverse them.

29 Infix to Postfix

Stack is used to convert infix expressions to postfix.

Operator precedence:

$$\boxed{^{>}*, / > +, -}$$

For example:

$$A + B * C$$

Postfix:

$$\boxed{ABC * +}$$

because multiplication has higher precedence.

Operator	Precedence
$**$	Highest
$*$, /, %	Medium
$+$, $-$	Lower

Associativity:

$$+, -, *, /, \%$$

are generally left-associative.
Exponentiation:
Exponentiation:

$$\boxed{\hat{}}$$

is commonly treated as a **right-associative** operator in expression-conversion questions.
is commonly treated as right-associative in expression-conversion questions.

30 Stack and Depth First Search

Depth First Search can be implemented using:

$$\boxed{\text{Stack}}$$

Either:

$$\boxed{\text{Explicit stack}}$$

or:

$$\boxed{\text{Recursion stack}}$$

DFS explores one path deeply before backtracking.

31 Stack and Backtracking

Backtracking problems often use stack-like behavior.
Examples:

- Maze solving
- N-Queens
- DFS
- Path exploration

The most recent decision is undone first.
Therefore:

$$\boxed{\text{LIFO}}$$

32 Numerical Practice 1

Question

An initially empty stack performs:

$$PUSH(10), PUSH(20), PUSH(30), POP(), PUSH(40)$$

What is the final top element?

After pushes:

$$[10, 20, 30]$$

POP removes:

$$30$$

Then:

$$PUSH(40)$$

Final:

$$[10, 20, 40]$$

Therefore:

$$\boxed{40}$$

33 Numerical Practice 2

Question

A stack contains:

$$[5, 10, 15, 20]$$

where 20 is TOP.
Two POP operations are performed.
What is the final stack?

First POP:

$$20$$

Second POP:

$$15$$

Final stack:

$$\boxed{[5, 10]}$$

TOP:

$$\boxed{10}$$

34 Numerical Practice 3

Question

A stack has capacity 10. Initially it is empty.
How many PUSH operations can be performed before overflow?

Maximum elements:

$$\boxed{10}$$

Therefore:

$$\boxed{10\ PUSH\ operations}$$

assuming no POP operations occur.

35 Numerical Practice 4

Question

For an array stack, $TOP = 6$.
How many elements are currently present?

Using:

$$\text{Number of elements} = TOP + 1$$

we get:

$$6 + 1 = 7$$

Therefore:

$$\boxed{7}$$

36 Numerical Practice 5

Question

An array stack has capacity 8. What is the value of TOP when the stack is full using zero-based indexing?

The last valid index is:

$$8 - 1 = 7$$

Therefore:

$$\boxed{TOP = 7}$$

37 Stack Trace – GATE Style

Consider:

$$PUSH(1)$$

$$PUSH(2)$$

$$PUSH(3)$$

$$POP()$$

$$PUSH(4)$$

$$POP()$$

$$POP()$$

Find the sequence of popped elements.
Initial:

$$[]$$

After PUSH operations:

$$[1, 2, 3]$$

POP:

$$\boxed{3}$$

Stack:

$$[1, 2]$$

PUSH(4):

$$[1, 2, 4]$$

POP:

$$\boxed{4}$$

Stack:

$$[1, 2]$$

POP:

$$\boxed{2}$$

Therefore:

$$\boxed{3,\ 4,\ 2}$$

38 GATE-Style Questions

Question 1

Which principle is followed by a stack?

A. FIFO

B. LIFO

C. Random access

D. Priority based

Answer:

$$\boxed{\text{B. LIFO}}$$

Question 2

Which operation removes the top element from a stack?

A. PUSH

B. POP

C. PEEK

D. TOP

Answer:

B. POP

Question 3

Which operation only reads the top element without removing it?

Answer:

PEEK

Question 4

For a standard stack, what is the time complexity of PUSH?

A. $O(1)$

B. $O(n)$

C. $O(\log n)$

D. $O(n^2)$

Answer:

$O(1)$

Question 5

What happens when POP is performed on an empty stack?

Answer:

Stack Underflow

Question 6

Which of the following applications naturally uses a stack?

A. Function calls

B. Undo operation

C. Parentheses matching

D. All of the above

Answer:

D. All of the above

Question 7

An array stack has capacity N. Using zero-based indexing, what is the TOP value when the stack is full?

Answer:

$N - 1$

Question 8

If $TOP = 9$ in an array stack using zero-based indexing, how many elements are present?

$$TOP + 1 = 10$$

Answer:

10

Question 9

Which data structure is most directly associated with DFS?

Answer:

Stack

DFS can use an explicit stack or recursion.

Question 10

Evaluate the postfix expression:

$$2\ 3\ +\ 4\ *$$

First:

$$2 + 3 = 5$$

Then:

$$5 \times 4 = 20$$

Therefore:

$$\boxed{20}$$

39 Stack vs Queue

Property	Stack	Queue
Principle	LIFO	FIFO
Insertion	TOP	REAR
Deletion	TOP	FRONT
Main operation	PUSH	ENQUEUE
Deletion operation	POP	DEQUEUE
Typical application	DFS	BFS

Remember:

$$\boxed{\text{Stack} \rightarrow LIFO}$$

$$\boxed{\text{Queue} \rightarrow FIFO}$$

40 Stack vs Array

An array is a general data structure supporting indexed access.

A stack is an abstract data type with restricted access.

For example:

Array:

$$A[0], A[1], A[2], \ldots$$

can be accessed directly.

Stack:

$$\boxed{\text{Only TOP is normally accessed}}$$

Therefore:

$$\boxed{\text{Stack is an ADT}}$$

and can be implemented using arrays or linked lists.

Important Concept

Stack is an Abstract Data Type (ADT).
Possible implementations include:

Array

and

Linked List

The LIFO behavior is what defines the stack, not the underlying implementation.

41 Important Results for GATE DA

Must Remember

Stack follows LIFO

PUSH inserts at TOP

POP removes from TOP

PEEK reads TOP without removing

Empty array stack: $TOP = -1$

Full stack of size N : $TOP = N - 1$

Number of elements $= TOP + 1$

PUSH $= O(1)$

POP $= O(1)$

PEEK $= O(1)$

Empty POP $\Rightarrow$ Underflow

Full PUSH $\Rightarrow$ Overflow

DFS $\Rightarrow$ Stack

BFS $\Rightarrow$ Queue

42 Practice Set – GATE DA Level

1. Define LIFO.
2. Explain PUSH and POP.
3. What is stack overflow?
4. What is stack underflow?
5. If $TOP = 5$, how many elements are present?
6. A stack has capacity 20. What is TOP when it is full?
7. Perform:
$$PUSH(5), PUSH(10), POP(), PUSH(20).$$
Find the final stack.
8. What is returned by PEEK?
9. Does PEEK remove an element?
10. What is the complexity of PUSH?
11. What is the complexity of POP?
12. Which data structure is naturally used for DFS?
13. Which data structure is naturally used for BFS?
14. Evaluate:
$$5\ 6\ +\ 2\ *$$
15. Evaluate:
$$8\ 2\ /\ 3\ +$$
16. Is a stack an ADT?
17. Give two implementations of a stack.
18. Why does recursion use a stack?
19. Explain stack overflow in recursion.
20. Convert:
$$A + B * C$$
to postfix.

43 Practice Answers

1. $\boxed{\text{Last In, First Out}}$
2. PUSH inserts; POP removes.
3. Trying to insert into a full stack.
4. Trying to remove from an empty stack.
5. $5 + 1 = \boxed{6}$
6. $\boxed{19}$
7. $[5, 20]$
8. Top element.
9. No.
10. $\boxed{O(1)}$
11. $\boxed{O(1)}$
12. Stack.
13. Queue.
14. $5 + 6 = 11$

    $11 \times 2 = 22$

    $\boxed{22}$
15. $8/2 = 4$

    $4 + 3 = 7$

    $\boxed{7}$
16. Yes.
17. Array and linked list.
18. Function calls are maintained in a call stack.
19. Excessive recursion can exhaust the available call stack.
20. $\boxed{ABC*+}$

44 30-Second GATE DA Revision

STACK – FINAL REVISION

STACK = LIFO

$PUSH \rightarrow$ Insert

$POP \rightarrow$ Delete

$PEEK \rightarrow$ Read TOP

$TOP = -1 \rightarrow$ Empty

$TOP = N - 1 \rightarrow$ Full

$\#Elements = TOP + 1$

$PUSH = O(1)$

$POP = O(1)$

$PEEK = O(1)$

Empty POP $\rightarrow$ Underflow

Full PUSH $\rightarrow$ Overflow

DFS $\rightarrow$ Stack

BFS $\rightarrow$ Queue

Recursion $\rightarrow$ Call Stack

Parentheses Matching $\rightarrow$ Stack

Postfix Evaluation $\rightarrow$ Stack

Infix to Postfix $\rightarrow$ Stack

GATE DA DSA – STACK COMPLETE

Next: Queue $\rightarrow$ Linked List $\rightarrow$ Trees